

Sky World Limited – Software Engineering Intern Pre-Interview Task

Overview

Build a Survey Platform that allows administrators to create and manage surveys while allowing users to respond to those surveys through a web or mobile application.

The platform should support dynamic survey creation and management, meaning surveys and questions should be configurable through the application and should not require code changes when new surveys or questions are added.

The task is split into three components:

1. Database
2. REST API
3. User Interface

Functional Requirements

The application should support the following capabilities.

Survey Management

Administrators should be able to:

- Create surveys
- Edit surveys
- View surveys
- Delete surveys

Question Management

Administrators should be able to:

- Add questions to a survey
- Edit questions
- Delete questions

The solution must support the following question types:

- Short Text
- Long Text
- Email
- Single Choice
- Multiple Choice
- File Upload

For choice-based questions, administrators should be able to:

- Add options
- Edit options
- Remove options

Survey Responses

Users should be able to:

- View available surveys
- Complete surveys
- Upload certificates where required
- Submit responses

Response Management

Administrators should be able to:

- View submitted responses
- Search responses by email address
- Download uploaded certificates

1. Database

Using a Relational Database Management System (MySQL, PostgreSQL, Oracle, or Microsoft SQL Server):

Requirements

1. Design an Entity Relationship Diagram (ERD) for the application.
2. Implement the database and name it: `sky_survey_db`

Deliverables

- ERD Diagram
- Database SQL Script

2. REST API

Using your preferred programming language, create a REST API that connects to your database.

The API responses illustrated in this document are expressed in XML and should be exposed to API consumers in XML format.

Internally, candidates are free to use their preferred data structures and serialization mechanisms provided the API contract remains consistent.

Survey Endpoints

Create Survey

URL

```
POST /api/surveys
```

Sample Request

```
<survey>
  <name>Graduate Developer Application Survey</name>
  <description>
    Initial candidate screening survey
  </description>
</survey>
```

Fetch Surveys

URL

```
GET /api/surveys
```

Sample Response

```
<surveys>
  <survey id="1">
    <name>Graduate Developer Application Survey</name>
    <description>
      Initial candidate screening survey
    </description>
  </survey>
  <survey id="2">
    <name>Internship Application Survey</name>
    <description>
      Internship application survey
    </description>
  </survey>
</surveys>
```

Question Endpoints

Create Question

URL

```
POST /api/surveys/{surveyId}/questions
```

Sample Request

```
<question name="full_name" type="short_text" required="yes">
  <text>What is your full name?</text>
  <description>[Surname] [First Name] [Other Names]</description>
</question>
```

Example Choice Question

```
<question name="programming_stack" type="choice" required="yes">
  <text>What programming stack are you familiar with?</text>
  <description>You can select multiple</description>
  <options multiple="yes">
    <option value="REACT">React JS</option>
    <option value="ANGULAR">Angular</option>
    <option value="VUE">Vue JS</option>
    <option value="JAVA">Java</option>
    <option value="PHP">PHP</option>
    <option value="GO">Go</option>
  </options>
</question>
```

Fetch Survey Questions

URL

```
GET /api/surveys/{surveyId}/questions
```

Sample Response

```
<questions>
  <question id="1" name="full_name" type="short_text" required="yes">
    <text>What is your full name?</text>
    <description>
      [Surname] [First Name] [Other Names]
    </description>
  </question>
  <question id="2" name="email_address" type="email" required="yes">
    <text>What is your email address?</text>
    <description/>
  </question>
  <question id="3" name="gender" type="choice" required="yes">
    <text>What is your gender?</text>
    <description/>
    <options multiple="no">
      <option value="MALE">Male</option>
      <option value="FEMALE">Female</option>
      <option value="OTHER">Other</option>
    </options>
  </question>
  <question name="programming_stack" type="choice" required="yes">
    <text>What programming stack are you familiar with?</text>
    <description>You can select multiple</description>
    <options multiple="yes">
      <option value="REACT">React JS</option>
      <option value="ANGULAR">Angular JS</option>
      <option value="VUE">Vue JS</option>
      <option value="SQL">SQL</option>
      <option value="POSTGRES">Postgres</option>
      <option value="MYSQL">MySQL</option>
      <option value="MSSQL">Microsoft SQL Server</option>
      <option value="JAVA">Java</option>
      <option value="PHP">PHP</option>
      <option value="GO">Go</option>
      <option value="RUST">Rust</option>
    </options>
  </question>
  <question name="certificates" type="file" required="yes">
    <text>Upload any of your certificates?</text>
    <description>You can upload multiple (.pdf)</description>
    <file_properties
      format=".pdf"
      max_file_size="1"
      max_file_size_unit="mb"
      multiple="yes"
    />
  </question>
</questions>
```

Survey Response Endpoints

Submit Survey Response

URL

```
POST /api/surveys/{surveyId}/responses
```

Requirements

- Support multipart/form-data

- Support multiple PDF uploads
- Persist all submitted responses

Sample Response Payload

```
<question_response>
  <full_name>Jane Doe</full_name>
  <email_address>janedoe@gmail.com</email_address>
  <description>I am an experienced Frontend Engineer.</description>
  <gender>FEMALE</gender>
  <programming_stack>REACT,VUE</programming_stack>
  <certificates>
    <certificate>
      Adobe Certification.pdf
    </certificate>
    <certificate>
      Figma Fundamentals.pdf
    </certificate>
  </certificates>
  <date_responded>
    2026-06-05 12:30:12
  </date_responded>
</question_response>
```

Fetch Survey Responses

URL

```
GET /api/surveys/{surveyId}/responses
```

Requirements

- Pagination
- Filtering by email address

Example Request

```
/api/surveys/1/responses?page=1&pageSize=10&email=jane@gmail.com
```

Sample Response Payload

```
<question_responses current_page="1" last_page="1" page_size="10" total_count="2">
  <question_response>
    <response_id>1</response_id>
    <full_name>John Doe</full_name>
    <email_address>johndoe@gmail.com</email_address>
    <description>I am an experienced FullStack Engineer with over 2 years experience. </description>
    <gender>MALE</gender>
    <programming_stack>REACT, JAVA, SQL, POSTGRES</programming_stack>
    <certificates>
      <certificate id="1">
        Oracle Java Certification 19-08-2023.pdf
      </certificate>
      <certificate id="2">
        Oracle SQL Certification 19-08-2023.pdf
      </certificate>
    </certificates>
    <date_responded>
      2023-09-21 12:30:12
    </date_responded>
  </question_response>
  <question_response>
    <response_id>2</response_id>
    <full_name>Jane Doe</full_name>
    <email_address>
      janedoe@gmail.com
    </email_address>
    <description>I am an experienced FrontEnd Engineer with over 6 years experience.</description>
    <gender>MALE</gender>
    <programming_stack>REACT, VUE</programming_stack>
    <certificates>
      <certificate id="3">
        Adobe Certification 19-08-2023.pdf
      </certificate>
      <certificate id="4">
        Figma Fundamentals 19-08-2023.pdf
      </certificate>
    </certificates>
    <date_responded>
      2023-09-23 12:30:12
    </date_responded>
  </question_response>
</question_responses>
```

Download Certificate

URL

```
GET /api/certificates/{id}
```

Requirements

Download a certificate using its unique identifier.

Postman Collection

Provide a Postman Collection documenting:

- All endpoints
- Sample requests
- Saved responses

3. User Interface

- Candidates should implement the survey and question management on web.
 - Candidates may choose to survey participation experience (survey discovery, survey completion, and response submission) on either web or mobile application.
-

Web Application Pages

Page 1: Survey Management (web)

Administrators should be able to:

- Create surveys
 - Edit surveys
 - Delete surveys
 - View surveys
-

Page 2: Question Management (web)

Administrators should be able to:

- Add questions
 - Edit questions
 - Delete questions
 - Manage options for choice questions
-

Page 3: Available Surveys (mobile application or web)

Users should be able to:

- View available surveys
 - View survey details
 - Select a survey to complete
-

Page 4: Survey Form (mobile application or web)

The page will allow users to respond to survey questions.

Requirements

1. Questions should be fetched dynamically from the API.
2. Implement a stepped form with one question per step.
3. Include Previous and Next navigation.
4. Hide Previous on the first step.
5. Validate required questions before proceeding.
6. Display a final review page before submission.
7. Submit responses using the Survey Response API.
8. Use the appropriate control for each question type.

Dynamic Form Requirement

The frontend must render survey questions dynamically using the metadata returned by the API. Adding a new question should not require frontend code changes.

Page 5: Survey Responses (web)

Administrators should be able to:

- View responses for a survey
- View paginated results
- Filter responses by email address
- Download uploaded certificates

Documentation

Each repository should contain a README.md that includes:

- Prerequisites
 - Installation instructions
 - Running locally
 - Technologies used
 - Assumptions made
-

Submission Requirements

Create public GitHub repositories.

1. **simple-survey-api**: repository for the api

This will contain:

- ERD Diagram
- Database SQL Script
- REST API Source Code
- Postman Collection

2. **simple-survey-web**: repository for the web application

This will contain:

- Web Application Source Code

3. **simple-survey-mobile** (*if implemented*): repository for the mobile application

This will contain:

- Mobile Application Source Code

Each repository should include:

- Meaningful commit history
 - README documentation
-

Deployment (Optional)

Additional credit will be awarded for:

- Deploying the API
 - Deploying the Web Application
 - Publishing the Mobile Application APK
 - Providing public URLs
 - Implementing both web and mobile applications
-

Timeline

Complete and submit the project within **two (2) weeks**.

Submission

Submit the following:

1. Link to the API repository
2. Link to the Web repository
3. Public deployment URL(s) (optional)
4. Link to the Mobile repository (if mobile application is implemented)
5. Mobile Application APK (if mobile application is implemented)

Ensure all links are publicly accessible and functional.

Be innovative, demonstrate sound engineering practices, and have fun building. Good luck!